

EVALUACIÓN · 4 APARTADOS

GRUPO 3 · MAYORO

Informe Técnico de Estrategia y Planificación de Pruebas

Sistema MAYORO · Plataforma B2B para Bodegas y Distribuidores Mayoristas

Curso: Pruebas y Calidad de Software

Objetivo: demostrar, siguiendo los 4 apartados de la evaluación, que la estrategia se aplicó, se implementó y produjo evidencia ejecutable para decidir la salida.

MANUEL FABRIZIO CALLAÑAUPA · CARLOS EDUARDO LOAIZA · RODRIGO EDUARDO CUSSE · PAUL GONZALO ACURIO

¿Qué es MAYORO y qué problema resuelve?

- Plataforma B2B que conecta bodegas con distribuidores mayoristas mediante catálogo, stock y cotización trazable.
- Reduce informalidad fiscal y errores comerciales: valida RUC de 11 dígitos, unicidad, IGV y cantidades mínimas de compra (MOQ).
- **Flujo completo:** registro comercial B2B → distribuidores publican productos → catálogo público consulta stock → MOQ controla cantidades → cotización codificada sale por WhatsApp.
- **Reglas críticas:** RUC válido/único identifica al negocio; IGV protege el total fiscal; roles separan acciones; stock evita prometer disponibilidad inexistente.
- **Estado de desarrollo:** funcional e intermedio: el flujo B2B principal está implementado y probado; quedan fuera carga/estrés, pagos y DAST integral.
- **Stack:** Laravel 12/PHP 8.2+ (rutas, controladores, Eloquent y validación) · MySQL (persistencia) · Blade/Tailwind (interfaz) · PHPUnit + Mockery (pruebas y dobles).



¿Qué entra en la entrega y qué queda fuera?

INCLUIDO

- **Eloquent + RUC:** valida 11 dígitos y unicidad para no duplicar identidades fiscales.
- **WhatsAppService:** normaliza teléfono, codifica el mensaje y rechaza entradas inválidas.
- **MOQ, IGV y totales:** bloquea cantidades insuficientes y verifica cálculo/redondeo.
- **Imágenes y stock:** sanitiza nombres, confina rutas y comunica estados reales.
- **Pruebas:** unitarias con Mockery; integración con RefreshDatabase y Storage::fake().

REGLAS, DATOS, CATÁLOGO, STOCK, IMÁGENES Y RBAC.

EXCLUIDO

- **Carga/estrés:** fuera por límites de XAMPP/Laragon; faltan infraestructura y métricas de concurrencia.
- **HU-05 Premium/pagos:** en refinamiento; no hay aún flujo estable ni pasarela definida.
- **DAST integral:** queda para otra iteración; aquí se cubre seguridad funcional, no escaneo dinámico completo.

SE PRIORIZA SEGURIDAD FUNCIONAL Y REGLAS DE NEGOCIO.

¿Cómo se combinan las pruebas para cubrir el riesgo?

Nivel	Tipo	Componente	Qué se verifica
Unitaria	Funcional	WhatsAppService Quotation totals	URL RFC 3986, IGV, redondeo y MOQ.
Unitaria	No funcional / seguridad	ProductImageStorage	Sanitización, extensiones y borrado seguro.
Integración / BDD	Funcional	UserRegistrationB2B ProductStockState	RUC, duplicidad, estados de catálogo y stock.
Integración / BDD	Seguridad	RBAC	Permisos y respuestas HTTP 403 ante IDOR.

ORDEN: UNITARIAS CRÍTICAS → INTEGRACIÓN/BDD (BD, RUTAS Y PERMISOS) → REGRESIÓN/SQA. **ACEPTACIÓN:** RESULTADO ESPERADO, EVIDENCIA REPRODUCIBLE Y 0 FAILED.

¿Qué riesgos críticos deben quedar controlados?

Prioridad	Riesgo	Mitigación / prueba	Resultado
Crítico	RUC inválido o duplicado	UserRegistrationB2BTest · regex de 11 dígitos + unicidad	Registro rechazado
Crítico	Path Traversal	ProductImageStorageTest · sanitización y borrado confinado	Ruta insegura bloqueada
Alto	Precio/cantidad menor que MOQ	QuotationServiceTest · regla comercial antes de cotizar	Cotización bloqueada
Alto	IDOR y bypass de roles	ProductPublishingNavigationTest · autorización HTTP	HTTP 403

CRITERIO: SE PRIORIZAN RIESGOS CRÍTICOS Y ALTOS PORQUE COMPROMETEN IDENTIDAD FISCAL, SEGURIDAD, DINERO O DISPONIBILIDAD; CADA PRUEBA DEBE FALLAR DE FORMA OBSERVABLE ANTES DE LA SALIDA.

¿Cómo se planifica, prioriza y traza la salida?

CRITERIO DE ACEPTACIÓN

0 FAILED

133 pruebas · 399 aserciones

6 defectos corregidos · PSR-12
con Laravel Pint.

ENTRADA: HISTORIAS/REGLAS
CLARAS Y DATOS PREPARADOS.
ORDEN: JIRA → RAMA FEATURE
→ PR → PHPUNIT/SQA → MERGE
A DEVELOP. **SALIDA:** 0
FAILED, EVIDENCIA REVISADA
Y CAMBIOS INTEGRADOS.

RESPONSABILIDADES

Paul Acurio · HU-01 / SCM:
registro y coordinación.

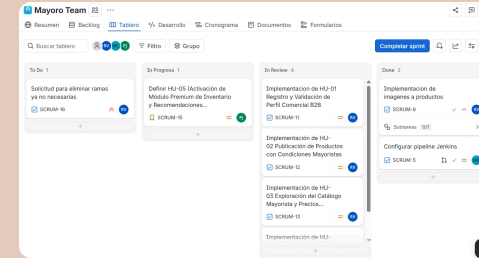
Rodrigo Cussi · HU-04, HU-02
Images: unitarias y mocks.

Manuel Callañaupa · HU-02,
HU-03 Stock: BDD y estados.

Carlos Loaiza · HU-03, HU-05
Premium: integración y flujo.

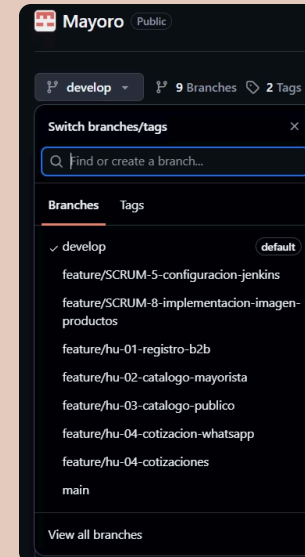
FLUJO: JIRA → RAMA FEATURE
→ PULL REQUEST →
PHPUNIT/SQA → MERGE A
DEVELOP.

Jira · seguimiento



TO DO · IN PROGRESS · IN REVIEW · DONE; SCRUM-5, 8, 11, 12, 13, 15 Y 16.

GitHub · trazabilidad



DEVELOP SELECCIONADA, 9 RAMAS Y 2 TAGS; EVIDENCIA
REPOSITORIO.

¿Cómo se traduce una regla en una prueba ejecutable?

M01 · USERREGISTRATIONB2BTEST

```
public function test_ruc_11_digits_and_unique(): void
{
    $response = $this->post('/register', [
        'ruc' => '20123456789',
        'email' => 'compras@mayoro.pe',
    ]);
    $response->assertStatus(302);
    $this->assertDatabaseHas('users', ['ruc' =>
'20123456789']);
    $this->post('/register', $data)->assertSessionHasErrors();
}
```

ENTRADA: RUC DE 11 DÍGITOS, CORREO ÚNICO Y DATOS SANITIZADOS. **RESULTADO ESPERADO:** REDIRECT + PERSISTENCIA; DUPLICADO/INVÁLIDO PRODUCE ERROR. **RELEVANCIA:** PROTEGE IDENTIDAD FISCAL Y CONVIERTE EL RIESGO CRÍTICO EN REGRESIÓN AUTOMATIZADA.

M02 · PRODUCTSTOCKSTATETEST

```
#[DataProvider('stockStates')]
public function test_catalog_state(
    int $stock, string $label
): void {
    $product = Product::factory()->create(
        ['stock' => $stock]
    );
    $this->get('/catalog')
        ->assertSee($label);
}
```

ENTRADA: PRODUCTO CON STOCK PARAMETRIZADO. **RESULTADO ESPERADO:** ETIQUETA CORRECTA Y “COTIZAR PEDIDO” SOLO SI SE PUEDE VENDER. **RELEVANCIA:** EVITA PROMETER DISPONIBILIDAD; MATRIZ IMPLEMENTADA: DISPONIBLE, TEMPORALMENTE SIN STOCK Y PAUSADO.

¿Cómo se protegen WhatsApp e imágenes?

U01 · WAMELINKGENERATORTEST

```
$url = $service->generate(
    '+51987654321',
    'Cotización: 2 cajas & IGV 18%'
);
$this->assertSame(
    'https://wa.me/51987654321?text=' .
    rawurlencode($message),
    $url
);
$this->expectException(InvalidArgumentException::class);
```

RFC 3986: TELÉFONO INVÁLIDO GENERA EXCEPCIÓN.

U02 · PRODUCTIMAGESTORAGETEST

```
$name = $storage->sanitize('../avatar.png');
$this->assertStringNotContainsString('..', $name);
$this->assertFalse(
    $storage->delete('../../.env')
);
Storage::fake('public');
```

Evidencia ejecutada

```
PS D:\WAYORO\Wayoro> php artisan test tests\Unit\ProductImageStorageTest.php

PASS Tests\Unit\ProductImageStorageTest
✓ sanea el nombre del cliente with data set "acentos y mayusculas" 0.19s
✓ sanea el nombre del cliente with data set "espacios en blanco" 0.05s
✓ sanea el nombre del cliente with data set "guiones repetidos" 0.03s
✓ sanea el nombre del cliente with data set "caracteres especiales" 0.02s
✓ sanea el nombre del cliente with data set "el punto se descarta y no se vuelve guion" 0.02s
✓ sanea el nombre del cliente with data set "solo simbolos" 0.02s
✓ sanea el nombre del cliente with data set "solo guiones" 0.02s
✓ sanea el nombre del cliente with data set "solo espacios" 0.02s
✓ sanea el nombre del cliente with data set "nombre vacio" 0.02s
✓ dos subidas del mismo archivo no se pisan 0.02s
✓ el archivo almacenado cae en la carpeta de productos 0.02s
✓ trunca los nombres excesivamente largos 0.02s
✓ la extension real manda sobre la declarada 0.02s
✓ url devuelve null para entradas vacias 0.01s
✓ url construye la direccion publica 0.02s
✓ eliminar borra el archivo previo 0.02s
✓ eliminar ignora rutas con traversal sin lanzar excepcion with data set "salto al directorio superior" 0.01s
✓ eliminar ignora rutas con traversal sin lanzar excepcion with data set "salto multiple" 0.01s
✓ eliminar ignora rutas con traversal sin lanzar excepcion with data set "salto combinado con ruta valida" 0.01s
✓ eliminar ignora rutas con traversal sin lanzar excepcion with data set "ruta absoluta de windows" 0.01s
✓ eliminar acepta null y cadena vacia 0.01s
✓ eliminar ignora un archivo inexistente 0.01s
✓ las constantes reflejan el criterio de aceptacion 0.01s

Tests: 23 passed (42 assertions)
Duration: 0.72s

% PS D:\WAYORO\Wayoro>
```

PRODUCTIMAGESTORAGETEST · CAPTURA REAL.

```
PS D:\WAYORO\Wayoro> php artisan test --filter=test_sanea_el_nombre_del_cliente

PASS Tests\Unit\ProductImageStorageTest
✓ sanea el nombre del cliente with data set "acentos y mayusculas"
✓ sanea el nombre del cliente with data set "espacios en blanco"
✓ sanea el nombre del cliente with data set "guiones repetidos"
✓ sanea el nombre del cliente with data set "caracteres especiales"
✓ sanea el nombre del cliente with data set "el punto se descarta y no se vuelve guion"
✓ sanea el nombre del cliente with data set "solo simbolos"
✓ sanea el nombre del cliente with data set "solo guiones"
✓ sanea el nombre del cliente with data set "solo espacios"
✓ sanea el nombre del cliente with data set "nombre vacio"

Tests: 9 passed (18 assertions)
Duration: 0.56s

PS D:\WAYORO\Wayoro>
```

FILTRO DE NOMBRES · CAPTURA REAL.

¿Cómo se valida la cotización antes de salir por WhatsApp?

U01 · ENLACE

RFC 3986

Codifica espacios, saltos de línea, acentos y símbolos para que WhatsApp reciba exactamente el mensaje generado.

U03 · TOTALES

IGV y redondeo

Verifica subtotal, IGV del 18% y total final a dos decimales con precios y cantidades controlados.

REGLA COMERCIAL

MOQ

Bloquea cantidad < MOQ y permite cantidad >= MOQ antes de emitir la cotización.

LA PRUEBA COMBINA REGLA, DATOS, ACCIÓN, ASERCIÓN Y EVIDENCIA DE EJECUCIÓN.

U01 · evidencia ejecutada

```
PS D:\MAYORO\Mayoro> php artisan test tests\Unit\WamelinkGeneratorTest.php
PASS Tests\Unit\WamelinkGeneratorTest
✓ genera enlace wa me con espacios codificados en por veinte
✓ codifica los saltos de línea como por veinte a
✓ normaliza telefono local de nueve digitos con prefijo 51
✓ valida el telefono como destino de whatsapp
✓ codifica acentos y símbolos reservados en utf8
Tests: 5 passed (16 assertions)
Duration: 0.13s
PS D:\MAYORO\Mayoro>
```

WAMELINKGENERATORTEST: CAPTURA REAL.

¿Qué demuestra la ejecución y qué se debe defender?

REPORTE ARTISAN / PHPUNIT

PASSED

0 FAILED · 133 pruebas · 399 aserciones

SUITES DESTACADAS: IMÁGENES 23 · WHATSAPP 5 ·
REGISTRO 4 · NAVEGACIÓN 13 · STOCK 23. LOS
PARES 23/42, 5/16 Y 23/82 REPRESENTAN
PRUEBAS/ASERCIONES POR SUITE; MUESTRAN
PROFUNDIDAD, NO PORCENTAJE DE COBERTURA.

DESARROLLO · 5 HALLAZGOS VISIBLES / 6 DEFECTOS
CORREGIDOS · FINAL: 0 FAILED

1. Namespace duplicado en ExampleTest: la suite volvió a cargar.
2. Login B2B: email vs email_contacto alineado.
3. Stock inicial corregido para partir del estado esperado.
4. Path Traversal bloqueado: evita HTTP 500.
5. Edición cruzada / IDOR bloqueada con HTTP 403.

LECTURA TÉCNICA: LOS DEFECTOS OCURRIERON DURANTE EL DESARROLLO Y QUEDARON CORREGIDOS; LA EJECUCIÓN FINAL REPORTA PASSED/0 FAILED. CADA PASS DEJA CONTROLADO UN RIESGO CONCRETO, SIN INVENTAR FALLOS FINALES NI PORCENTAJE DE COBERTURA.

¿Qué aprendimos y cuál es la siguiente iteración?

CONCLUSIÓN

TDD + BDD

Combinar pruebas de unidad con escenarios integrados en Laravel conecta la regla técnica con el comportamiento que el negocio necesita.

DIFICULTAD ENCONTRADA

Aislamiento y trazabilidad

Mockery, RefreshDatabase y Storage::fake() exigieron separar dependencias, datos y archivos para conservar evidencia reproducible sin contaminar el entorno.

ITERACIÓN 2

- PCOV/Xdebug para cobertura por líneas.
- E2E con Laravel Dusk o Playwright.
- Lock optimista para concurrencia de stock.

UTILIDAD: LA ESTRATEGIA CONECTA RIESGO, REGLA, PRUEBA Y DECISIÓN. **MEJORAS:** AMPLIAR COBERTURA MEDIDA, E2E Y CONCURRENCIA EN LA SIGUIENTE ITERACIÓN; AVANZAR AHORA CON EVIDENCIA CONTROLADA.